QUICKSHOW

Complete Website Flow, Backend Endpoints & Transaction Design

Reconstructed Project Blueprint

This document is the revised implementation blueprint for QuickShow – Online Movie Ticket Booking and Theatre Management System. It preserves the original project requirements and architecture while incorporating the required corrections for search, authentication, show-seat initialization, concurrency, booking/payment consistency, cancellation and server-side validation.

> Core architecture
> Thymeleaf + HTML/CSS/JS → Spring Boot Controller → Service → JdbcTemplate Repository + raw SQL → MySQL

> Core rule
> Browsing can be public, but booking, payment, profile and cancellation require an authenticated customer. Seat availability is authoritative at ShowSeatAllocates level.

Layer	Approach
Frontend	Thymeleaf + HTML/CSS/JS
Backend	Spring Boot
Controller	Endpoint-focused controllers using ResponseEntity
Service	Business validation, transactions and repository coordination
Repository	JdbcTemplate + explicit raw SQL only
Database	MySQL
Authentication	Spring Security; customer/admin role protection

1. Overall Architecture

```
Browser / Thymeleaf page
↓
Controller
↓
Service
↓
Repository (JdbcTemplate + raw SQL)
↓
MySQL
```

HTTP response convention

Situation	Response
Successful GET	200 OK
Successful POST / resource creation	201 CREATED
Successful PUT	200 OK
Successful DELETE	204 NO CONTENT
Missing resource	404 NOT FOUND
Invalid request	400 BAD REQUEST
Seat/booking/business conflict	409 CONFLICT
Unauthenticated	401 UNAUTHORIZED / redirect for Thymeleaf login
Forbidden admin/customer operation	403 FORBIDDEN

Implementation rule

2. Database-to-Website Functional Model

```
Movie
↓
Show
↓
Screen
↓
Seat
↓
ShowSeatAllocates
↓
Booking
↓
BookedSeats
↓
Payment
```

Object	Website usage
Movie	Home, movie list/details, search/filter, show selection, admin movie management
MovieCast	Movie details and admin cast management
Theatre	Theatre list/details, show discovery, admin theatre management
Screen	Theatre details, show selection, seat selection, admin screen management
Seat	Seat layout, selection and admin seat configuration
Show	Movie details, show selection, seat selection and admin scheduling
ShowSeatAllocates	Show-specific availability and concurrency-safe seat booking
Customer / CustomerEmail	Registration, login, profile and booking ownership

Object	Website usage
Booking / BookedSeats	Summary, confirmation, My Bookings and cancellation
Payment	Checkout result, confirmation and booking/payment history
Admin	Admin login and protected management

3. Revised Customer Website Journey

```
HOME / LANDING
↓
Browse Movies / Search / Filter
↓
Movie Details
↓
Select Show
↓
Seat Selection
↓
Booking Summary
↓
Payment / Final Checkout
↓
Booking Confirmation
↓
My Bookings
↓
Booking Details / Cancellation
```

> Authentication boundary
> Guests may browse movies, theatres, shows and seat availability. When a guest attempts to book, redirect to login. After successful login, return the customer to the intended booking flow.

Customer pages

Page	URL	Main functionality
Home	/	Movie search, now showing, upcoming movies, cards, ratings, login/register
Movie Listing	/movies	Catalogue, title search, language/genre/certificate/rating filters
Movie Details	/movies/{movieId}	Movie information, cast, theatres, screens and available shows
Theatre Listing	/theatres	Browse theatres by location
Theatre Details	/theatres/{theatreId}	Address, screens, screen type, capacity and shows
Login	/login	Customer/admin authentication and routing
Registration	/register	Create Customer + CustomerEmail
Profile	/profile	View/edit profile, password and booking access
Show Selection	/movies/{movieId}/shows	Choose date, theatre, screen, time and show
Seat Selection	/shows/{showId}/seats	Seat layout, availability, selected seats and calculated total
Booking Summary	/shows/{showId}/booking	Review show, seats, price and total before checkout
Payment	/payment	Choose/confirm payment method and complete final checkout
Confirmation	/booking/{bookingId}/confirmation	Booking, seats, amount and payment status
My Bookings	/bookings	Upcoming/past bookings and statuses
Booking Details	/bookings/{bookingId}	Full details and cancellation where allowed

4. Movie Discovery and Search Flow

```
GET /api/movies
↓
Optional GET /api/movies/search?title=&language;=&genre;=&certificate;=&minRating;=
↓
Movie cards
↓
```

```
GET /api/movies/{movieId}
↓
GET /api/movies/{movieId}/cast
↓
GET /api/movies/{movieId}/shows
↓
Show selection
```

Required search correction
The original requirements explicitly require title search and filtering by language, genre and certification. The revised endpoint supports these together; minimum IMDb rating is an optional additional filter.

Recommended movie endpoints

Method	Endpoint	Purpose
GET	/api/movies	Get movies / basic listing
GET	/api/movies/{id}	Get movie by ID
GET	/api/movies/search?title=...	Search/filter by title and optional filters
GET	/api/movies/{movieId}/cast	Get movie cast
GET	/api/movies/{movieId}/shows	Get shows for movie
POST	/api/movies	Create movie – admin
PUT	/api/movies/{id}	Update movie – admin
DELETE	/api/movies/{id}	Delete movie if dependency rules allow – admin

5. Show Discovery and Selection

```
Movie Details
↓
GET /api/movies/{movieId}/shows
↓
Group shows by date / theatre
↓
Customer selects a specific ShowID
↓
GET /api/shows/{showId}
↓
GET /api/shows/{showId}/seats
↓
Seat Selection
```

Show information displayed

- Movie title
- Theatre name and location
- Screen name and screen type
- Show date and time
- Ticket price
- Available seat count
- Show status

Show creation – admin transaction

```
Admin submits movie + screen + date + time + price
↓
Validate Movie exists
↓
```

```
Validate Screen exists
↓
Validate no overlapping show on the same screen
↓
Create Show
↓
Read all seats belonging to the Screen
↓
Create one ShowSeatAllocates row per seat with AVAILABLE status
↓
Commit
```

> Important
> A Show must not become usable for customer seat selection until its ShowSeatAllocates rows have been initialized.

6. Seat Selection and ShowSeatAllocates

Concept	Rule
Seat	Physical seat belonging to one screen
ShowSeatAllocates	One row for a screen seat for a particular show
Status	AVAILABLE or BOOKED
Browser selection	Temporary UI state only; does not book a seat
Authoritative availability	Database ShowSeatAllocates status
Total amount	Calculated by server from show ticket price × selected seat count

```
GET /api/shows/{showId}/seats
↓
Display seat map
↓
Customer selects available seats in browser
↓
Browser stores selected SeatIDs temporarily
↓
Customer opens Booking Summary
↓
Server re-validates seats during final checkout
```

> Do not trust the browser
> A seat that was AVAILABLE when the page loaded may be BOOKED by another customer before checkout. The final checkout must re-check and atomically claim the seats.

7. Revised Booking and Payment Flow

```
Seat Selection
↓
Booking Summary
↓
Payment Page
↓
POST /api/bookings/checkout
↓
Authenticate customer
↓
Validate Show
↓
Validate Show is bookable
↓
```

```
Validate requested seats belong to the Show
↓
Atomically change AVAILABLE → BOOKED
↓
Create Booking
↓
Create BookedSeats
↓
Calculate and store total amount
↓
Process / record Payment
↓
Set Booking = CONFIRMED
↓
Update Show.AvailableSeats
↓
COMMIT
↓
Booking Confirmation
```

> Critical transaction rule
> The final booking operation must not leave partial state. If a required step fails, the transaction rolls back. This prevents Booking/BookedSeats/seat status/payment from becoming inconsistent.

Recommended public booking endpoint

Method	Endpoint	Purpose
POST	/api/bookings/checkout	Final authenticated checkout; coordinates booking, booked seats, seat allocation and payment
GET	/api/bookings/{id}	Get booking details after ownership/admin authorization
POST	/api/bookings/{id}/cancel	Cancel booking when business rules allow
GET	/api/customers/{customerId}/bookings	Customer's booking history; must be authorization-protected

8. Concurrent Booking Protection

Atomic seat claim

```
Customer A and Customer B request the same seat
↓
Server attempts conditional UPDATE
↓
UPDATE ShowSeatAllocates SET Status='BOOKED' WHERE ShowID=? AND ScreenID=? AND SeatID=? AND Status='AVAILABLE'
↓
Check rowsUpdated
↓
rowsUpdated = 1 → seat successfully claimed
↓
rowsUpdated = 0 → seat was already claimed → return 409 CONFLICT
```

> Repository responsibility
> Keep the concurrency-sensitive SQL in the repository. The service coordinates the complete transaction and handles the business outcome.

Why ShowSeatAllocates is authoritative

9. Cancellation Flow

```
My Bookings
↓
Booking Details
↓
Customer selects Cancel
↓
Authenticate customer
↓
Verify booking belongs to customer
↓
Verify Booking status = CONFIRMED
↓
Verify show has not started / cancellation rule permits cancellation
↓
Set Booking = CANCELLED
↓
Read BookedSeats
↓
Change corresponding ShowSeatAllocates → AVAILABLE
↓
Increase Show.AvailableSeats
↓
Update / record payment cancellation or refund status as applicable
↓
COMMIT
```

> Cancellation integrity
> Cancellation is also a multi-table transaction. A booking must not become CANCELLED while its seats remain BOOKED.

10. Registration, Login and Authorization

```
Registration form
↓
Validate customer data
↓
Validate email uniqueness
↓
Hash password
↓
Create Customer
↓
Retrieve generated UserID
↓
Create CustomerEmail
↓
COMMIT
↓
Login
↓
Spring Security authentication
↓
Customer / Admin role routing
```

Authentication boundaries

Access	Rule
Home / movies / theatres / shows / seat availability	Public

Access	Rule
Booking checkout	Authenticated customer
Payment / booking details	Authenticated customer; ownership checked
My Bookings	Authenticated customer; only own bookings
Cancellation	Authenticated customer; only own eligible bookings
Admin pages and management endpoints	Authenticated admin only

> Credential security
> Passwords must be stored as secure hashes, not plaintext. Customer profile responses must never expose the stored password.

11. Admin Website Journey

```
ADMIN LOGIN
↓
ADMIN DASHBOARD
↓
Movies
↓
Movie Cast
↓
Theatres
↓
Screens
↓
Seats
↓
Shows
↓
Bookings
↓
Customers
```

Admin pages

Page	URL	Functionality
Admin Login	/admin/login	Authenticate administrator
Dashboard	/admin/dashboard	Counts, statistics and management links
Movies	/admin/movies	List/add/edit/delete movies
Movie Cast	/admin/movies/{movieId}/cast	View/add/remove actors
Theatres	/admin/theatres	List/add/edit/delete theatres
Screens	/admin/theatres/{theatreId}/screens	Manage screens, types and capacity
Seats	/admin/screens/{screenId}/seats	Configure seats/layout
Shows	/admin/shows	Create/edit/delete shows; scheduling and prices
Bookings	/admin/bookings	View bookings, seats, customers, amounts and statuses
Customers	/admin/customers	View customer information and relevant bookings

Admin dependency-aware deletion

12. Complete Endpoint Map

Feature	Method	Endpoint
Movies	GET	/api/movies

Feature	Method	Endpoint
Movie	GET	/api/movies/{id}
Movie Search	GET	/api/movies/search?title=&language;=&genre;=&certificate;=&minRating;=
Movie Cast	GET	/api/movies/{movieId}/cast
Movie Cast	POST	/api/movies/{movieId}/cast
Movie Cast	DELETE	/api/movies/{movieId}/cast/{actor}
Movie Shows	GET	/api/movies/{movieId}/shows
Theatres	GET/POST/PUT/DELETE	/api/theatres...
Theatre Shows	GET	/api/theatres/{theatreId}/shows
Screens	GET/POST/PUT/DELETE	/api/screens...
Screen Seats	GET/POST/DELETE	/api/screens/{screenId}/seats...
Shows	GET	/api/shows/{id}
Shows	POST/PUT/DELETE	/api/shows...
Show Seats	GET	/api/shows/{showId}/seats
Customer	POST	/api/customers/register
Customer	GET/PUT	/api/customers/{id}
Auth	POST	/api/auth/login
Bookings	POST	/api/bookings/checkout
Booking	GET	/api/bookings/{id}
Customer Bookings	GET	/api/customers/{customerId}/bookings
Cancellation	POST	/api/bookings/{id}/cancel
Payment	GET	/api/payments/{id}
Admin	Protected	All management endpoints above

13. Controller → Service → Repository Responsibilities

Controller	Service responsibilities	Main repositories
MovieController	Validation, search parameters, movie use cases	MovieRepository, MovieCastRepository
TheatreController	Theatre validation and management	TheatreRepository
ScreenController	Theatre ownership, capacity/type validation	ScreenRepository
SeatController	Screen ownership and seat configuration	SeatRepository
MovieCastController	Validate movie and cast changes	MovieCastRepository
ShowController	Validate movie/screen, overlap checks, create allocations	ShowRepository, SeatRepository, ShowSeatAllocationRepository
CustomerController	Registration/profile/email/password operations	CustomerRepository, CustomerEmailRepository
BookingController	Authenticated checkout, ownership, cancellation	BookingRepository, BookedSeatsRepository, ShowSeatAllocationRepository, ShowRepository, PaymentRepository
PaymentController	Payment lookup / controlled payment operations	PaymentRepository
Admin/Auth	Authentication and role protection	Spring Security + relevant repositories

14. Recommended Response / View Models

DTO	Useful fields
MovieDetailsDTO	Movie fields + cast + show summaries
ShowSummaryDTO	ShowID, date, time, price, status, theatre, screen
SeatAvailabilityDTO	ShowID, ScreenID, SeatID, row, seat number, status

DTO	Useful fields
BookingSummaryDTO	BookingID, show, seats, seat count, price, total amount
BookingDetailsDTO	Booking + booked seats + payment status + movie/theatre/screen/show
CustomerProfileDTO	Customer profile fields + emails; never password

15. Essential Business Validation Rules

- Movie title, language, duration and release date must satisfy database/application constraints.
- A Show must reference an existing Movie and Screen.
- A Show cannot overlap another show on the same Screen.
- ShowSeatAllocates must be initialized from the Screen's actual seats.
- A requested booking seat must belong to the Show's Screen.
- Only AVAILABLE ShowSeatAllocates can be claimed.
- Seat claiming must be atomic under concurrency.
- TotalSeatCount is derived from the selected seat IDs.
- TotalAmount is calculated server-side from the Show ticket price.
- Only authenticated customers can checkout.
- A customer can access/cancel only their own bookings.
- Only eligible CONFIRMED bookings can be cancelled.
- Cancellation restores the corresponding seats and available-seat count.
- Admin management endpoints require admin authorization.
- Deletes must respect dependent records and database integrity.
- Passwords are hashed before persistence.

16. Transaction Boundaries

Operation	Transaction contents
Customer registration	Create Customer + generated UserID + CustomerEmail
Create Show	Create Show + initialize all ShowSeatAllocates
Checkout	Validate + claim seats + Booking + BookedSeats + Payment + confirmation state + AvailableSeats
Cancellation	Validate + Booking CANCELLED + allocations AVAILABLE + AvailableSeats + payment/refund status
Admin delete	Delete only when dependency rules permit; use transaction where multiple records change

Atomicity goal
A failure after any required database mutation must roll back all mutations belonging to the same business operation.

17. Final Website Blueprint

```
QUICKSHOW
↓
PUBLIC
↓
Home → Movies → Search/Filter → Movie Details → Shows → Seat Availability
↓
↓
↓
Select ShowID
↓
↓
↓
Select Seats
```

```
↓
↓
↓
Booking Summary
↓
↓
↓
Login if needed
↓
↓
↓
Payment / Checkout
↓
↓
↓
Server validation + atomic seat claim
↓
↓
↓
Booking + BookedSeats + Payment
↓
↓
↓
COMMIT
↓
↓
↓
Confirmation
↓
↓
↓
My Bookings
↓
↓
↓
Booking Details
↓
↓
↓
Cancellation
↓
↓
↓
Seats AVAILABLE again
```

Parallel admin flow

```
Admin Login
↓
Dashboard
↓
Movies / Cast
↓
Theatres
↓
Screens
↓
```

```
Seats
↓
Shows
↓
ShowSeatAllocates initialized automatically
↓
Bookings
↓
Customers
```

18. Recommended Implementation Order

Step	Feature	Backend flow
1	Movies	MovieController → MovieService → MovieRepository
2	Movie Cast	MovieCastController → MovieCastService → MovieCastRepository
3	Theatres	TheatreController → TheatreService → TheatreRepository
4	Screens	ScreenController → ScreenService → ScreenRepository
5	Seats	SeatController → SeatService → SeatRepository
6	Shows	ShowController → ShowService → ShowRepository + allocation initialization
7	ShowSeatAllocates	Allocation service/repository + concurrency-safe updates
8	Customer / Email	CustomerController → CustomerService → repositories
9	Security	Spring Security authentication + customer/admin authorization
10	Booking	BookingController → BookingService → Booking/BookedSeats/Allocation repositories
11	Payment	Payment coordination within checkout + PaymentRepository
12	Cancellation	BookingService transaction restoring seats
13	Thymeleaf	Connect completed endpoints to customer/admin pages
14	Testing	Repository → service → controller → concurrent booking → browser/database verification

19. End-to-End Verification Scenarios

Scenario	Expected result
Search movie by title	Matching movies returned
Filter by language/genre/certificate	Only matching movies returned
Open movie details	Movie + cast + shows visible
Select a show	Specific ShowID carried to seat page
Open seat page	Every screen seat has show-specific status
Create show as admin	Show and all ShowSeatAllocates created atomically
Book available seats	Booking, booked seats, payment and BOOKED allocations committed
Two customers book same seat	Only one succeeds; other receives 409 CONFLICT
Payment/checkout failure	No partial booking or permanently BOOKED seats
Cancel booking	Booking CANCELLED and seats AVAILABLE again
Customer views bookings	Only own bookings returned
Customer accesses another booking	403/404 according to chosen security policy
Admin accesses management	Allowed only for admin
Guest attempts checkout	Redirect/login or 401
Delete referenced movie/screen/etc.	Rejected with dependency conflict when appropriate

20. Final Implementation Checklist

- Movie title search added.
- Language, genre and certificate filters supported.
- ShowID used as the central booking-flow identifier.
- Show creation initializes ShowSeatAllocates.
- Show overlap validation implemented.
- Seat availability is determined from ShowSeatAllocates.
- Final checkout recalculates price and seat count server-side.
- Atomic AVAILABLE → BOOKED seat update implemented.
- 409 CONFLICT returned for unavailable seats.
- Booking, BookedSeats, Payment and seat state are transactionally consistent.
- Cancellation restores seats and available-seat count.
- Customer ownership checks are enforced.
- Spring Security protects customer/admin routes.
- Passwords are securely hashed.
- Admin deletes respect dependency rules.
- DTOs are used for combined view/API responses.
- Repository layer remains JdbcTemplate + raw SQL only.
- Repository, service, controller and end-to-end tests cover critical paths.